Feature demo · the guards register

Cut the tail, keep the shape — and still say so.

`guards: strict` lets an overflowing slide end in an ellipsis instead of shearing mid-line. It does far less than the name suggests, on purpose: every slide after this one shows a limit rather than a win.

What it does: the card keeps its shape.

Short card

One short line.

Short card two

Another short line here.

The oversized card

This body is written long enough to force its row taller than its neighbor and push the grid past the frame, which is exactly the shape the guard exists for: the grid stays two-by-two, the body ends in an ellipsis, and the reader sees a finished card rather than a sheared one. A reviewer looking at four cards should not have to

Short card four

The last short line — stretched to match its tall row-mate.

Content clipped

What a number never gets: an ellipsis.

A slot's trim class is a property of what the text means, not of the component. An ellipsis on a sentence says "there is more". An ellipsis on `$1,234,567` states a different number, and the failure is silent — the deck looks better than the truth.

So headings, KPI values, code, math, citations, legal text and footers are never trimmed. Anything unclassified defaults to never, which is why `strict` does less rather than something wrong.

It declines rather than cut where nobody can see the mark.

If the block that crosses the frame edge sits wholly below it, clamping that block puts the ellipsis off-screen: the content is gone, the mark is invisible, and the slide renders looking finished. Measured across the component gallery, a prototype without this rule did exactly that on nineteen slides in eighty-one.

So the guard declines and leaves the honest clip. A slide that looks complete and is missing two paragraphs is worse than one that visibly overflows, because only the second one tells you to fix it.

This slide is that case, live. It overflows, and the guard declines: the block crossing the frame edge starts below it, so any ellipsis placed there would be drawn where you cannot see it. The plan refuses to put a mark nobody can see, nothing is cut, and that is why you are reading a slide that clips rather than one quietly missing its tail.

Content clipped

The alarm survives the guard, because it is a different alarm.

The overflow ring is geometric, so a guard that works turns it off — that is the guard working, not a signal being lost. The "Content clipped" tag reads text rects, and a clamp leaves its lines laid out, so a trimmed slide still reports.

What the existing probes cannot see is a removal, so a trim stamps its own record and the export prints a `✂ TRIMMED` line naming the pages. Inheriting the alarm was this feature's first design, and it was wrong.

The honest number.

On the decks this repo ships, `guards: strict` resolves **one of the six** slides that clip today. The other five are blocked by a heading, a callout box's own chrome, or a shell command one line-height from the frame edge.

That is the shape of the feature: the blocks that most often cause overflow are largely the ones it refuses to touch. It is a guard on the tail of a paragraph, not a fix for a slide with too much on it.

guards: loose is the default

Nothing changes unless you ask for it.

A deck that omits the key renders exactly as it did before. Per-slide, `<!-- _class: guards-loose -->` *takes one slide back out.*